

Sorting & Algorithm Analysis — Summary Notes

1. Types of Sorting

Comparison-Based Sorting

Algorithms: **Bubble, Selection, Insertion, Merge Sort, Quick Sort, Randomized Quick Sort**

- Compare pairs of elements and decide whether to swap.
- Lower bound for comparison-based sorting: $\Omega(N \log N)$.
- Typical complexities:
 - Bubble / Selection / Insertion: $O(N^2)$
 - Merge: $O(N \log N)$
 - Quick Sort (avg): $O(N \log N)$, (worst): $O(N^2)$

Non-Comparison Sorting

Algorithms: **Counting Sort, Radix Sort**

- Do **not** compare elements.
- Can beat $N \log N$ lower bound under assumptions.
- Counting sort: $O(N + k)$
- Radix sort: depends on digits, $O(d \times (N + \text{base}))$

2. Iterative vs Recursive

- **Recursive sorting**: Merge Sort, Quick Sort, Randomized Quick Sort
- Typically use **Divide and Conquer (D&C)**:
 - Divide problem into subproblems
 - Solve recursively
 - Combine results

3. Algorithm Analysis Basics

Operation Counting

- Count # of operations to assess efficiency
- Example complexities:
 - $O(N^2)$
 - $O(N \log N)$
 - $O(N)$

Asymptotic Analysis

- Focus on **large n**
- Only consider **leading term**

- Ignore constants
 - e.g., $100n^2 \rightarrow n^2$

Growth Rates (from fastest to slowest)

Time Complexity	Name
$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(n)$	Linear
$O(n \log n)$	Quasilinear
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(2^n)$	Exponential
$O(n!)$	Factorial
∞	Unbounded

4. Sorting Algorithms (Details)

4.1 Bubble Sort

- For $i = 0 \dots n$, compare adjacent elements; swap if needed
- Continue until no more swaps
- Worst/Average: $O(N^2)$
- In-place
- Stable

4.2 Selection Sort

Procedure:

1. Set $L = 0$
2. Find smallest element from $L \dots n$
3. Swap with element at L
4. Increase L

- $O(N^2)$
- In-place
- **Not stable**
- Few swaps (good for memory-write-limited systems)

4.3 Insertion Sort

- Start at index 1
 - Insert each element into the correct position on the left (like sorting cards)
 - Best: **$O(N)$** (when nearly sorted)
 - Avg/Worst: **$O(N^2)$**
 - In-place
 - Stable
-

5. $O(N \log N)$ Comparison Sorts

5.1 Merge Sort

- Pure D&C
- Split $\rightarrow$ Sort $\rightarrow$ Merge
- Each level: **$O(N)$**
- Depth: **$O(\log N)$**
- Total: **$O(N \log N)$**

Properties:

- **Stable**
 - **Not in-place** (needs $O(N)$ memory)
 - Always $O(N \log N)$ regardless of input order
-

5.2 Quick Sort

- Choose pivot
- Partition array around pivot
- Recursively sort left & right

Runtime

Case	Complexity	Reason
Best	$O(N \log N)$	Balanced splits
Avg	$O(N \log N)$	Random input
Worst	$O(N^2)$	Already sorted array, or bad pivot choice

Why worst case happens

- Pivot chosen as leftmost element
- Sorted input $\rightarrow$ all elements go to the right partition
- Depth becomes $O(N)$

Notes

- In-place

- **Not stable**
 - Faster than merge sort in practice due to cache locality
-

5.3 Randomized Quick Sort

- Pivot chosen randomly
 - Probability of worst-case extremely small
 - Expected complexity: **$O(N \log N)$**
 - Good practical performance
-

6. Non-Comparison Sorting

6.1 Counting Sort

Assumptions

- Elements are integers
- Range $0 \dots k$ is small

Method

1. Count frequency of each value
2. Compute prefix sums (optional for stable version)
3. Output values in order

Complexity

- **$O(N + k)$**
 - If k is too large $\rightarrow$ impractical
 - Naive version: **not stable**
 - With prefix sums: **stable**
-

6.2 Radix Sort

Assumptions

- Elements have few digits (e.g., integers, strings)
- Digit range small

Method (LSD version)

1. For each digit from least significant $\rightarrow$ most significant
2. Place values into 10 buckets (0–9)
3. Recombine buckets

Properties

- **Stable**
 - Usually implemented using stable Counting Sort as subroutine
 - Complexity: $O(d \times (N + \text{base}))$
-

7. Other Sorting Properties

Stability

- Stable sorting keeps the relative order of items with the same key
- Examples: Merge sort, Insertion sort, Counting sort (stable version), Radix sort
- Non-stable: Quick sort, Selection sort, Heap sort

In-place

- Uses only $O(1)$ extra space
- In-place: Bubble, Selection, Insertion, Quick sort
- Not in-place: Merge sort, Counting sort (usually), Radix sort

Caching / Memory Locality

- Quick sort has better cache performance (local partitioning)
- Merge sort uses extra memory → may have more cache misses